DNS Notes: [DOMAIN]

AUTHOR: MUSTAFA-ALMOHSEN

1️⃣ CNAME Record
2️⃣ A Record (IPv4)
3️⃣ AAAA Record (IPv6)
4️⃣ MX Records (Mail Servers)
5️⃣ NS Records (Nameservers)
6️⃣ SOA Record (Start of Authority)
7️⃣ TXT Records (SPF / DMARC / DKIM)
7️⃣- Zone transfer
DNS potential attack vectors / misconfigurations.
1️⃣ **Zone Transfer (AXFR) Vulnerability**

2️⃣ **DNSSEC (Domain Name System Security Extensions)**

3️⃣ **SPF (Sender Policy Framework)**

4️⃣ **DKIM (DomainKeys Identified Mail)**

6️⃣ **Wildcard DNS Records**

7️⃣ **Subdomain Enumeration**

9️⃣ **Nameserver Configuration**
🔎 What is DNS Rebinding?
⚡ What this means:
🛡️ Defenses (for your notes/reporting)
🔎 Step-by-step with my theory
⚠️ Key Notes
🔥 DNS Rebinding Workflow in Bug Bounties
1. Find an SSRF
2. Basic internal probing
3. Deploy DNS rebinding domain

4. Use SSRF with rbndr domain

1️⃣ CNAME Record

Command:

```
dig CNAME [DOMAIN]
```

Result:

```
```

Notes:

- Present / Not present
- If present, alias → canonical name

```
```

2️⃣ A Record (IPv4)

Command:

```
dig A [DOMAIN]
```

Result:

```
```

Notes:

- IPv4 addresses hosting the domain

3️⃣ AAAA Record (IPv6)

Command:

```
dig AAAA [DOMAIN]
```

Result:

```
```

Notes:

- IPv6 addresses if available

4️⃣ MX Records (Mail Servers)

Command:

```
dig MX [DOMAIN]
```

Result:

```
```

Notes:

- Mail servers & priority
- SPF/DKIM/DMARC compliance

5️⃣ NS Records (Nameservers)

Command:

```
dig NS [DOMAIN]
```

Result:

```
```

Notes:

- Authoritative nameservers
- Often shows registrar (e.g., GoDaddy, Cloudflare)

6️⃣ SOA Record (Start of Authority)

Command:

```
dig SOA [DOMAIN]
```

Result:

```
```

Notes:

- Primary nameserver, admin email, TTL values

7️⃣ TXT Records (SPF / DMARC / DKIM)

Command:

```
dig TXT [DOMAIN]
```

Result:

```
```

Notes:

- Email authentication, verification record

7️⃣- Zone transfer

Command:

```
dig axfr Domaim  @NS
```

Result:

```
```

Notes:

-

DNS potential attack vectors / misconfigurations.

AUTHOR: MUSTAFA-ALMOHSEN

1️⃣ Zone Transfer (AXFR) Vulnerability

```
dig axfr example.com @nsztm1.digi.ninja
```

Expected Output

```
; <<>> DiG 9.10.6 <<>> axfr example.com @nsztm1.digi.ninja
;; global options: +cmd
;; connection timed out; no servers could be reached
```

- **Allowed:** If the output lists DNS records (e.g., A, MX, TXT), the zone transfer is permitted.
- **Denied:** If you receive a timeout or no response, the zone transfer is restricted.

Risk: Zone transfers should be disabled to prevent unauthorized access to all DNS records, which can aid in further attacks.

2️⃣ DNSSEC (Domain Name System Security Extensions)

```
dig +dnssec example.com
```

Expected Output:

```
; <<>> DiG 9.10.6 <<>> +dnssec example.com
;; global options: +cmd
;; flags: qr rd ra ad
;; QUESTION SECTION:
;example.com.            IN   A
;; ANSWER SECTION:
example.com.         3600    IN   A    93.184.216.34
example.com.         3600    IN   RRSIG    A 13 2 3600 202309271
80434 20230925160434 35273 example.com. DYYZ/bhHSAIlpvu/HEUsx
lzkC9NsswbCQ7dcfcuiNBrbhYV7k3AI8t46 QMnOlfhwT6jqsfN7ePV6Fwpym
3B0pg==
```

Interpretation:

- **Enabled:** The presence of the `ad` flag in the header and `RRSIG` records in the answer section indicates DNSSEC is active.
- **Disabled:** Absence of the `ad` flag or `RRSIG` records suggests DNSSEC is not configured.

Risk: Without DNSSEC, the domain is vulnerable to DNS spoofing and cache poisoning attacks.

3️⃣ SPF (Sender Policy Framework)

```
dig txt example.com
```

Expected Output:

```
example.com.        3600    IN  TXT "v=spf1 include:_spf.google.com ~all"
```

Interpretation:

- **Configured:** An SPF record is present, specifying which mail servers are permitted to send emails on behalf of the domain.
- **Missing or Misconfigured:** Absence of this record or a weak policy (e.g., `~all` instead of `all`) can lead to email spoofing.

Risk: Weak or absent SPF records increase the likelihood of phishing and spam attacks.

4️⃣ DKIM (DomainKeys Identified Mail)

```
dig txt selector1._domainkey.example.com
```

Expected Output:

```
selector1._domainkey.example.com. 3600 IN TXT "v=DKIM1; k=rsa; p=MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA7kD5O+g5zU8zFZ5VZ5J4..."
```

Interpretation:

- **Configured:** A DKIM record is present, allowing receiving mail servers to verify the authenticity of the sender.

- **Missing:** Absence of this record means emails cannot be cryptographically verified, increasing the risk of impersonation.

Risk: Missing DKIM records make the domain susceptible to email spoofing.

6️⃣ Wildcard DNS Records

```
dig any *.example.com
```

Output:

```
*.example.com.      3600    IN  A   93.184.216.34
```

Interpretation:

- **Present:** A wildcard record exists, directing all subdomains to a single IP address.
- **Absent:** No wildcard record is configured.

Risk: Wildcard records can mask unused or unconfigured subdomains, potentially leading to subdomain takeover if not properly managed.

7️⃣ Subdomain Enumeration

```
dig axfr example.com @nsztm1.digi.ninja
```

Output

```
; <<>> DiG 9.10.6 <<>> axfr example.com @nsztm1.digi.ninja
;; global options: +cmd
```

```
;; connection timed out; no servers could be reached
```

Interpretation:

- **Allowed:** If the output lists subdomains, they are publicly accessible.
- **Denied:** If you receive a timeout or no response, subdomains are not publicly listed.

Risk: Exposed subdomains can reveal internal services or misconfigurations, increasing the attack surface.

9️⃣ Nameserver Configuration

```
dig ns example.com
```

Output

```
example.com.        3600    IN  NS  ns1.exampledns.com.
example.com.        3600    IN  NS  ns2.exampledns.com.
```

Interpretation:

- **Configured:** The domain has authoritative nameservers set up.
- **Misconfigured:** If the nameservers point to expired or unresponsive servers, DNS resolution may fail.

Risk: Misconfigured nameservers can lead to domain unavailability.

🔎 What is DNS Rebinding?

It's an attack that tricks a victim's web browser into bypassing the **same-origin policy (SOP)** by abusing how DNS resolution works.

🟢 **Normal DNS (safe)**

```
$ dig test.com

;; ANSWER SECTION:
test.com.       300    IN    A    93.184.216.34
```

- TTL = **300 seconds** (5 minutes)
- IP is stable
- No risk of rebinding.

🔴 **Suspicious DNS (potential rebinding)**

```
$ dig rebindattacker.com

;; ANSWER SECTION:
rebindattacker.com.    1    IN    A    1.2.3.4
```

- TTL = **1 second**
- That means next time the browser queries again, it must re-resolve.
- If the attacker's DNS changes IPs, the victim's browser will follow.

🔄 **After TTL expires (second lookup)**

```
$ dig rebindattacker.com

;; ANSWER SECTION:
rebindattacker.com.    1    IN    A    192.168.0.1
```

- Same hostname → **different IP**
- This time the IP points to the victim's internal router

- Browser still thinks it's "the same origin" (`rebindattacker.com`), but traffic goes to `192.168.0.1`.

⚡ What this means:

- The **low TTL** + **IP switch** between lookups = **classic DNS rebinding behavior**.
- That's how attacker JS in a browser pivots from their server to your internal network.

🛡️ Defenses (for your notes/reporting)

1. **Browser protections** – Chrome/Firefox block rebinding to `localhost`, some block private ranges.
2. **DNS resolvers** – Some resolvers refuse to serve mixed public/private responses.
3. **Web app firewalls** – Can restrict Host headers or origin checks.
4. **Developers** – Don't rely on client-side isolation; enforce server-side authentication

---

🔎 Step-by-step with my theory

1. **Recon target public IP**
   - Yes, normally you'd `dig target.com` or resolve the subdomain the app is hosted on.
   - Example:

     ```
     dig +short target.com
     34.201.100.55
     ```

- Now you know what IP the application really sits on (AWS / GCP / Azure / whatever).

2. **Configure rbndr (A/B switching)**
    - On the rebinder service (https://lock.cmpxchg8b.com/rebinder.html

```
7ab43c2d.7f000001.rbndr.us
```

3. **Victim interacts with rbndr domain**

- You somehow need the *victim's browser or server-side code* to connect to that rbndr domain instead of the real one.
- On **first DNS lookup**, it resolves to the real public IP (works fine, app sees nothing wrong).
- On **next DNS lookup**, rbndr flips it → now `169.254.169.254` (AWS metadata).

4. **Attacker's goal**

- The browser/server is tricked into sending HTTP requests *thinking it's still talking to target.com*, but now it's really sending traffic to AWS metadata.
- If Same-Origin Policy or SSRF protections are weak, you can get:
    - AWS tokens (`/latest/meta-data/iam/security-credentials/role-name`) Internal secrets (e.g. GCP metadata tokens)

⚠️ Key Notes

- Without SSRF, rebinding is hard to apply on random targets.
- With SSRF + IP filter, rebinding is **the classic bypass**.
- Works not only for AWS, but also:
    - Internal web apps (10.x / 192.168.x ranges)
    - Redis, Memcached, Elasticsearch, etc.
    - Cloud metadata (AWS, GCP, Azure, DigitalOcean)

🔥 DNS Rebinding Workflow in Bug Bounties

1. Find an SSRF

I find a parameter like:

```
https://target.com/fetch?url=http://example.com
```

- It fatch remote resources and i confiremed SSRF exist here well.

2. Basic internal probing

Next, you try:

- `http://127.0.0.1:80`
- `http://169.254.169.254/latest/meta-data/` (AWS)
- `http://metadata.google.internal/computeMetadata/v1/` (GCP)

If it blocks *private IPs* or *special hosts*, your SSRF impact looks limited.

`This is where DNS rebinding comes in.`

3. Deploy DNS rebinding domain

Go to rbndr

- Field **A** = any public IP
- Field **B** = target internal IP (`169.254.169.254`)

It gives you:

```
abcd1234.rbndr.us
```

4. Use SSRF with rbndr domain

Now feed SSRF this domain:

```
https://target.com/fetch?url=http://abcd1234.rbndr.us/latest/meta-data/
```

- On first time abcd1234.rbndr.us resolver it to external IP not blocked, the App will allow it cuz its sees an external IP.
- Second time or may third, fouth,Fifth time abcd1234.rbndr.us will resolve it to (`169.254.169.254`). Now the request will go to Meta data aws services which Bypass the Filters. Result we retrive the external Meta data of AWS.